

Ynara — Plan de Fine-Tuning sobre Modelos Open-Source

Modelos base: Gemma 4 26B-A4B (conversacional) + Qwen 3.5-9B (agente), ambos Apache 2.0.

Hardware: RTX 4080 Super (16 GB VRAM).

Método elegido en el informe técnico: Unsloth + QLoRA.

Documento: Mayo 2026. Punto de partida técnico para el equipo.

Resumen ejecutivo

El BMC compromete fine-tuning sobre pesos abiertos de Hugging Face, y es la decisión correcta: es lo que convierte "Ynara usa modelos open-source" en "Ynara tiene una voz rioplatense que ningún competidor global va a priorizar". Pero hay una verdad técnica incómoda que el informe técnico no anticipó del todo y que cambia el plan:

La RTX 4080 Super (16 GB) puede fine-tunear cómodamente Qwen 3.5-9B, pero la Gemma 4 26B-A4B queda al borde o directamente afuera. La Gemma 4 26B-A4B es un modelo MoE, y el fine-tuning de MoE en QLoRA es problemático: la documentación oficial de Unsloth desaconseja QLoRA sobre MoE y un modelo MoE comparable (Qwen3-30B-A3B) ya necesita 17,5 GB — más de los 16 GB disponibles. Servir el modelo (inferencia) sí entra; entrenarlo es otra cosa.

Recomendación central: dividir la estrategia por modelo.

1. **Qwen 3.5-9B:** fine-tuning local en la 4080 Super con QLoRA. Viable, probado, barato.
2. **Gemma 4 26B-A4B:** fine-tuning en cloud puntual (alquiler de una GPU de 24-48 GB por unas horas, costo de decenas de dólares) o evaluar si realmente hace falta tunearla, dado que su fuerte (rioplatense conversacional) ya viene bien de fábrica.

Y un orden pragmático: **empezar por Qwen 3.5-9B**, que es donde el fine-tuning local es viable y de bajo riesgo, y donde el tool-calling rioplatense tiene más margen de mejora.

1. ¿Fine-tuning sí o no? (la pregunta previa)

Antes de gastar GPU y tiempo, vale preguntarse si el fine-tuning es necesario. La regla de la industria en 2026: **primero prompt engineering y few-shot, después fine-tuning**. El fine-tuning se justifica cuando ya probaste todos los trucos de prompting y el resultado está "cerca pero no del todo".

Para Ynara, el fine-tuning **sí se justifica**, por tres razones concretas:

1. **Voz rioplatense consistente.** El prompt puede pedir voseo, pero un modelo fine-tuneado lo internaliza: no se le escapan deslices peninsulares bajo presión, mantiene el registro a lo largo de conversaciones largas, y no gasta tokens de contexto en instrucciones de estilo.
2. **Tool-calling confiable en Qwen.** El modo Productividad depende de que Qwen arme JSON correcto sin alucinar. Fine-tunar sobre los formatos de herramientas específicos de Ynara mejora la confiabilidad medible.
3. **Es el moat de la tesis.** La memoria procedural rioplatense + un modelo tuneado para el registro argentino es exactamente lo que big tech no va a hacer primero. Sin fine-tuning, el pitch de "infra propia adaptada" se debilita.

Lo que **no** hay que fine-tunar: conocimiento general, capacidades de razonamiento (ya vienen), nada que se resuelva con un buen system prompt.

2. Modelo base: análisis de viabilidad por modelo

2.1 Qwen 3.5-9B (agente) — fine-tuning local viable

- **Arquitectura:** densa, 9B parámetros. Las arquitecturas densas son las más amigables para QLoRA.
- **VRAM en bf16 LoRA (dato oficial Unsloth):** 22 GB. No entra en 16 GB.
- **VRAM en QLoRA 4-bit:** aproximadamente la mitad, ~11-13 GB con gradient checkpointing. **Entra en la 4080 Super, con headroom ajustado.**
- **Veredicto:** fine-tuning local viable. Es el candidato ideal para empezar.
- **Cuidado:** secuencias largas (>2048 tokens) o batch size mayor a 1-2 pueden tirar out-of-memory. Mantener `max_seq_length=2048`, `batch_size=1-2`, `gradient_accumulation=4`, gradient checkpointing activado.

2.2 Gemma 4 26B-A4B (conversacional) — fine-tuning local problemático

- **Arquitectura:** MoE, 26B totales / 4B activos. Las arquitecturas MoE son las más difíciles para QLoRA.
- **VRAM en LoRA 16-bit (dato oficial Unsloth):** más de 40 GB. Imposible en 16 GB.
- **VRAM en QLoRA 4-bit:** la documentación de Unsloth **desaconseja QLoRA sobre MoE** y recomienda usar el modelo denso equivalente. Un MoE comparable (Qwen3-30B-A3B) necesita 17,5 GB — ya por encima de los 16 GB de la 4080 Super.
- **Problema adicional MoE:** no conviene entrenar la capa de router (Unsloth la deshabilita por defecto). Y el modelo 16-bit completo hay que descargarlo y convertirlo a 4-bit al vuelo, lo que pide RAM y disco extra.
- **Veredicto:** fine-tuning local en la 4080 Super es, en el mejor caso, extremadamente ajustado, y en el peor, inviable. Tres caminos:

1. **Cloud puntual:** alquilar una GPU de 24 GB (RTX 4090, ~USD 0,55/hora) o 48 GB (A6000/A100) por las horas que dure el entrenamiento. Costo estimado: decenas de dólares por corrida.
2. **No tunear Gemma:** dado que el fuerte de Gemma 4 (calidez, rioplatense, EQ) ya viene de fábrica, quizás alcance con prompt engineering + memoria procedural, y reservar el fine-tuning solo para Qwen.
3. **Tunear una variante densa más chica de Gemma 4** (si existe en la familia) que entre cómodo en 16 GB, y usarla en los modos conversacionales.

Recomendación honesta: arrancar sin fine-tunear Gemma. Validar si con prompting + memoria procedural el rioplatense conversacional es suficiente. Si no alcanza, hacer una corrida cloud puntual. No vale la pena pelear con MoE QLoRA en una 4080 Super cuando el costo cloud es de decenas de dólares.

2.3 Tabla de viabilidad

Modelo	Arquitectura	QLoRA en 4080 Super (16GB)	Recomendación
Qwen 3.5-9B	Densa	Sí, ~11-13 GB, ajustado	Fine-tuning local, empezar acá
Gemma 4 26B-A4B	MoE	No / al borde (~17,5 GB)	Cloud puntual o no tunear en MVP

3. Método: LoRA vs QLoRA vs full fine-tuning

Método	Qué hace	VRAM	Cuándo usarlo en Ynara
Full FT	Entrena todos los pesos	60-70 GB+ para 7B	Nunca en MVP. Inviabile en consumer hardware.
LoRA	Entrena adaptadores en 16-bit, base congelada en 16-bit	22 GB para 9B	Solo si se consigue GPU de 24 GB+
QLoRA	Base cuantizada a 4-bit + adaptadores	~11-13 GB para 9B	La elección para Ynara. Estándar 2026 en consumer.

Decisión: QLoRA, como ya marcaba el informe técnico. Es el estándar de la industria en 2026 para todo lo que esté por debajo de 34B en hardware consumer. Con Unsloth, QLoRA iguala la performance de full fine-tuning usando 4× menos VRAM.

Detalle de QLoRA que importa: se entrenan solo ~0,2% de los parámetros. El resultado es un adaptador chico (50-200 MB) que después se mergea con el modelo base o se sirve por separado. Esto significa que se pueden tener **varios adaptadores** (uno por registro, o experimentales) sin re-descargar el modelo base.

4. Dataset: la estrategia de datos (el cuello de botella real)

Acá está el problema del huevo y la gallina: para fine-tuneear necesitás datos rioplatenses de instrucción, pero Ynara todavía no tiene usuarios que los generen. Y el research confirma algo importante: **no existe un dataset rioplatense de instruction-tuning listo para usar en Hugging Face**. Hay datasets de audio argentino (TTS), datasets de detección de hate speech rioplatense, pero nada de conversación instruccional rioplatense general.

Esto es a la vez un problema y una oportunidad de tesis: construir ese dataset es contribución original.

4.1 Estrategia recomendada: datos sintéticos + curaduría

Tres fuentes combinadas:

Fuente 1 — Generación sintética con modelo teacher (núcleo de la estrategia).

Usar un modelo grande (puede ser una Gemma 4 grande, Qwen 3.5 grande, o incluso un modelo cloud potente solo para generar datos —no para servir— cuidando que la licencia permita usar las salidas para entrenar) para generar pares instrucción-respuesta en rioplatense. El proceso:

1. Definir los 5 modos y el tono de cada uno (ya está en `ynara.config.json` y `TONE-OF-VOICE.md`).
2. Generar prompts variados que un usuario argentino haría en cada modo.
3. Generar respuestas en rioplatense correcto, con voseo, registro adecuado por modo.
4. **Curaduría humana**: revisar, corregir deslices, eliminar lo que suena peninsular o robótico. Esta curaduría es lo que da calidad — un dataset sintético sin revisar es ruido.

Fuente 2 — Datasets públicos en español como base.

Tomar datasets de instrucción en español (existen varios generales) y adaptarlos al registro rioplatense vía reescritura. No para usarlos tal cual, sino como esqueleto de tareas que después se "argentinizan".

Fuente 3 — Datos reales (cuando haya usuarios, con consentimiento).

Post-launch, con el consentimiento opt-in que definimos en el informe de compliance, las conversaciones reales (anonimizadas) son el dataset más valioso. Esto cierra el flywheel: más usuarios → mejor dataset → mejor modelo → más usuarios.

4.2 Tamaño de dataset

Según las mejores prácticas 2026:

- **500-1.000 ejemplos**: tareas simples (formato, clasificación). Suficiente para un primer experimento de voz rioplatense.
- **3.000-10.000 ejemplos**: adaptación de dominio / instruction-following complejo. Es el rango target para un fine-tune serio de registro.
- **Más de 10.000**: rendimientos decrecientes salvo dominio muy amplio.

Recomendación para Ynara: arrancar con un piloto de ~1.000 ejemplos curados para validar el pipeline, después escalar a 3.000-5.000 para el fine-tune real de Qwen.

4.3 Formato

Formatos estándar 2026: **ShareGPT** (conversaciones multi-turno, ideal para Ynara), **ChatML**, o **Alpaca** (instrucción simple). Para un asistente conversacional con memoria, ShareGPT es el más natural.

Guardar como JSONL.

4.4 Reglas de calidad del dataset

- Deduplicar.
- Filtrar por longitud (eliminar ejemplos demasiado cortos o largos).
- Revisar manualmente al menos 100 ejemplos para detectar problemas de calidad.
- Split 80/20 train/eval.
- **La mayoría de los fine-tunes fallan por problemas de dataset, no de hardware.** Invertir el tiempo acá.

5. Hiperparámetros recomendados (Qwen 3.5-9B QLoRA en 4080 Super)

Punto de partida basado en los defaults de Unsloth para 2026, ajustado a los 16 GB:

```
from unsloth import FastModel

model, tokenizer = FastModel.from_pretrained(
    model_name = "unsloth/Qwen3.5-9B",
    max_seq_length = 2048,          # subir solo si sobra VRAM
    load_in_4bit = True,           # QLoRA
    full_finetuning = False,
)

model = FastModel.get_peft_model(
    model,
    r = 16,                        # rank: 16 para registro/estilo, 32 si hace falta más capacidad
    lora_alpha = 16,               # alpha = rank es un buen default
    lora_dropout = 0,
    bias = "none",
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                     "gate_proj", "up_proj", "down_proj"],
    use_gradient_checkpointing = "unsloth", # crítico para 16 GB
)
```

Config de entrenamiento:

```

from trl import SFTConfig

args = SFTConfig(
    per_device_train_batch_size = 1,      # 16 GB obliga a batch chico
    gradient_accumulation_steps = 4,      # batch efectivo = 4
    warmup_steps = 5,
    num_train_epochs = 2,                 # 1-3 epochs; más = overfitting
    learning_rate = 2e-4,                 # default sólido para LoRA
    logging_steps = 1,
    optim = "adamw_8bit",                 # optimizador 8-bit ahorra VRAM
    weight_decay = 0.01,
    lr_scheduler_type = "linear",
)

```

Notas:

- **rank (r):** para tareas de estilo/registro, 16 alcanza. Si el modelo no aprende suficiente, subir a 32. Más rank = más capacidad pero más VRAM y más riesgo de overfitting.
- **alpha = rank** es la regla práctica de Unsloth.
- **epochs:** 1-3. Para un dataset chico, 2 epochs es un buen punto. Más epochs sobreajusta y el modelo empieza a "loro-repetir".
- **gradient checkpointing** no es opcional en 16 GB — es lo que hace que entre.
- **batch size 1 + grad accumulation 4** es la combinación para no quedarse sin VRAM.

6. Métricas de éxito

Acá hay un problema honesto: **no existe un benchmark estándar de rioplatense para LLMs**. Hay que construirlo, y esa es otra contribución de tesis. Tres niveles de evaluación:

6.1 Eval automática de registro (construir)

Un set de 30-60 preguntas/prompts que disparen rioplatense, con criterios medibles:

- ¿Usa voseo consistentemente? (vos tenés, no tú tienes)
- ¿Evita peninsular? (sin "vale", "ordenador", "vosotros", "coger" en sentido peninsular)
- ¿Usa argentinismos naturalmente cuando corresponde, sin forzar?
- ¿Mantiene el registro a lo largo de una conversación larga?

Medir el modelo base vs el fine-tuneado sobre este set. Es la métrica más importante y la más original.

6.2 Eval de tool-calling (para Qwen)

- % de JSON válido generado
- % de herramientas correctas elegidas para una intención dada

- % de parámetros correctos

Comparar base vs fine-tuneado. Aquí Qwen ya parte de una base alta (BFCL-V4 = 66.1 según el informe), así que se mide si el fine-tune sobre los formatos de Ynara mejora o al menos no degrada.

6.3 Eval de no-regresión

Crítico: el fine-tune no puede romper lo que ya funcionaba. Verificar que el modelo fine-tuneado:

- No perdió capacidad de razonamiento general
- No perdió inglés (si se tuneó muy agresivo en español puede degradarse)
- No empezó a alucinar más

Un fine-tune que mejora rioplatense pero rompe el razonamiento es un fracaso. Si el dataset es muy chico o muy repetitivo, esto pasa. Por eso conviene mezclar algunos ejemplos de capacidades generales en el dataset (mantener al menos algo de diversidad).

6.4 Criterio de éxito concreto

El fine-tune se considera exitoso si: sube medible en la eval de registro rioplatense, mantiene o mejora tool-calling (Qwen), y no regresa en no-regresión. Si cualquiera de las tres falla, se itera el dataset o se hace rollback al modelo base.

7. Costo de cómputo estimado

7.1 Qwen 3.5-9B local (4080 Super)

- **Costo monetario directo:** cero (hardware propio).
- **Costo de electricidad:** despreciable para corridas de horas.
- **Tiempo estimado:** para referencia, un QLoRA de 7B con ~3.000 ejemplos, 2 epochs, seq 2048, corre en bajo una hora en una RTX 4090. La 4080 Super es algo más lenta (menos núcleos, menos ancho de banda), estimar **1,5-3 horas por corrida** para un dataset de 3.000-5.000 ejemplos. El 9B agrega un poco más que el 7B de referencia.
- **Iteraciones:** presupuestar varias corridas (ajustar dataset, rank, epochs). Total realista: un fin de semana de experimentación para tener un fine-tune sólido.

7.2 Gemma 4 26B-A4B cloud (si se decide tunear)

- **GPU:** alquiler de RTX 4090 (24 GB) a ~USD 0,55/hora, o A100/A6000 (48 GB) algo más caro pero con headroom.
- **Tiempo:** un MoE de este tamaño en QLoRA, varias horas. Estimar **4-8 horas** por corrida.
- **Costo por corrida:** en el rango de **USD 5-30** según GPU y duración.

- **Con 3-4 iteraciones:** total en el orden de **USD 30-100** para tener un fine-tune de Gemma. Asumible para una tesis.

7.3 Costo total estimado del plan de fine-tuning

Ítem	Costo
Qwen 3.5-9B local	~USD 0 (hardware propio) + tiempo de equipo
Gemma 4 26B-A4B cloud (opcional)	USD 30-100 si se decide tuneear
Generación de dataset sintético	Variable: si se usa un modelo cloud como teacher, costo de API; si se usa modelo local, gratis pero más lento
Total realista MVP	Menos de USD 150 en cómputo, más el tiempo de curaduría del dataset (que es el verdadero costo)

El costo real del fine-tuning no es la GPU, es la curaduría del dataset. Generar y limpiar 3.000-5.000 ejemplos rioplatenses de calidad es trabajo humano de varios días. Ahí está el grueso del esfuerzo.

8. Pipeline completo (de datos a modelo servido)

1. **Preparar entorno:** Python 3.11+, PyTorch 2.5+, CUDA 12.x, Unsloth, transformers, datasets, peft, trl, bitsandbytes. Pinear versiones para reproducibilidad.
2. **Generar dataset:** síntesis con modelo teacher → curaduría humana → formato ShareGPT JSONL → dedup + filtrado → split 80/20.
3. **Fine-tune Qwen 3.5-9B:** QLoRA con los hiperparámetros de la sección 5, en la 4080 Super.
4. **Evaluar:** correr las tres evals (registro, tool-calling, no-regresión) base vs fine-tuneado.
5. **Decidir:** si pasa los criterios, mergear el adaptador o servirlo. Si no, iterar dataset/hiperparámetros.
6. **Exportar:** a GGUF (para Ollama/llama.cpp en dev) o servir el adaptador con vLLM (producción).
7. **Versionar:** guardar el adaptador, el dataset, los hiperparámetros y los resultados de eval. Cada fine-tune es un experimento reproducible (esto va en `docs/architecture/adrs/` si cambia la decisión, o en un registro de experimentos).
8. **Gemma 4 (si aplica):** repetir 3-6 en cloud.

9. Roadmap por fases

Fase 0 — Validar sin fine-tuning (semanas 1-2)

Antes de tunear nada, probar si prompt engineering + memoria procedural alcanzan para el rioplatense. Construir la eval de registro (las 30-60 preguntas). Medir el baseline de Gemma y Qwen sin tunear. **Si el baseline ya es bueno, quizás no haga falta fine-tunear Gemma en absoluto.**

Fase 1 — Dataset piloto + primer fine-tune Qwen (semanas 3-5)

Generar ~1.000 ejemplos curados. Fine-tunear Qwen 3.5-9B local. Medir contra el baseline. Aprender qué funciona del pipeline.

Fase 2 — Dataset completo + fine-tune Qwen serio (semanas 6-9)

Escalar a 3.000-5.000 ejemplos. Fine-tune de producción de Qwen. Eval completa. Este es el entregable principal de fine-tuning para la tesis.

Fase 3 — Gemma cloud (opcional, si Fase 0 mostró que hace falta)

Corrida cloud puntual de Gemma 4 26B-A4B. Solo si el rioplatense conversacional sin tunear no alcanzó.

Fase 4 — Flywheel con datos reales (post-launch)

Con usuarios y consentimiento opt-in, incorporar conversaciones reales anonimizadas al dataset. Re-tunear periódicamente. Acá el moat se profundiza.

10. Caveats y decisiones abiertas

- **El plan asume que Qwen 3.5-9B es densa.** El informe técnico lo dice ("arquitectura densa de 9B"), lo que la hace tuneable local. Si fuera MoE, aplicaría el mismo problema que Gemma. Verificar al descargar los pesos.
- **MoE QLoRA es terreno movedizo.** La situación de fine-tuning de MoE en consumer hardware mejora mes a mes. Vale re-chequear la documentación de Unsloth cuando se llegue a la Fase 3 — puede que para entonces el 26B-A4B entre mejor en 16 GB.
- **Licencia de los datos sintéticos.** Si se usa un modelo cloud como teacher para generar el dataset, verificar que sus términos permitan usar las salidas para entrenar otro modelo. Algunos lo prohíben. Con un teacher open-source (Gemma/Qwen grande) este problema no existe.
- **El consentimiento para datos reales** (Fase 4) ya está cubierto en el informe de compliance — es opt-in, separado, revocable. No saltarlo: Replika tiene una investigación abierta justamente por entrenar con datos de usuarios sin base legal clara.

- **No sobre-tunear.** El riesgo más común es un dataset chico y repetitivo que hace que el modelo "olvide" capacidades generales. Mejor un dataset diverso de 3.000 ejemplos que uno monótono de 10.000.
 - **La 4080 Super es de 16 GB — el límite es real.** Todo el plan respeta ese techo. Si el equipo consiguiera una GPU de 24 GB (4090) o más, varias cosas se simplifican (Gemma local, batch más grande, secuencias más largas).
-

11. Conexión con el resto del proyecto

- **Informe técnico:** este plan implementa la "Capa LLM — Unsloth + QLoRA" y refina la suposición de que ambos modelos se tunean local. Corrección: Qwen local sí, Gemma probablemente cloud.
 - **Research de memoria:** el fine-tune de Qwen sobre los formatos de extracción de memoria (ADD/UPDATE/DELETE/NOOP) puede mejorar la calidad de la memoria semántica. Considerar incluir ejemplos de extracción en el dataset de Qwen.
 - **Compliance:** el flywheel de datos reales (Fase 4) depende del consentimiento opt-in que ya está diseñado. El fine-tuning y el compliance se tocan exactamente en ese punto.
 - **Repo:** el pipeline de fine-tuning vive en `apps/backend/` (o un paquete aparte tipo `ml/`), los experimentos se versionan, y cualquier cambio de decisión (cambiar de QLoRA a otra cosa, cambiar modelo base) va por ADR.
-

Documento generado como punto de partida técnico para el fine-tuning de Ynara. Mayo 2026. Las versiones de librerías y las capacidades de fine-tuning de MoE cambian rápido — verificar la documentación de Unsloth al momento de ejecutar cada fase.